

Security Audit

Haiku (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	20
Conclusion	21
Our Methodology	22
Disclaimers	24
About Hashlock	25

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Haiku team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Haiku (accessible at haiku.trade) is an AI-powered DeFi aggregator and intent-driven trading platform that lets users execute complex, multi-step strategies—such as swaps, bridges, lending, borrowing, and liquidity management—through simple natural language commands or declarative intents. It optimizes execution using a convex optimiser for minimal slippage and swift outcomes, all bundled into a single on-chain transaction. The platform supports multiple EVM-compatible networks (e.g., Arbitrum, Base, BNB Chain) and has undergone security audits, with the most recent one in February 2025.

Additionally, Haiku incorporates a token-based system centered around the \$HKU (an ERC-20 token), which powers the ecosystem and is tied to engagement via a Karma points and airdrop program. Users earn points through activity like executing intents, building referral networks, and participating in social/community challenges—structured across tiers (e.g., Pebble to Monk)—ahead of the Token Generation Event (TGE)

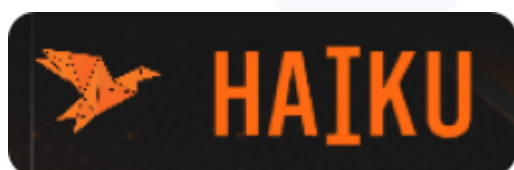
Project Name: Haiku

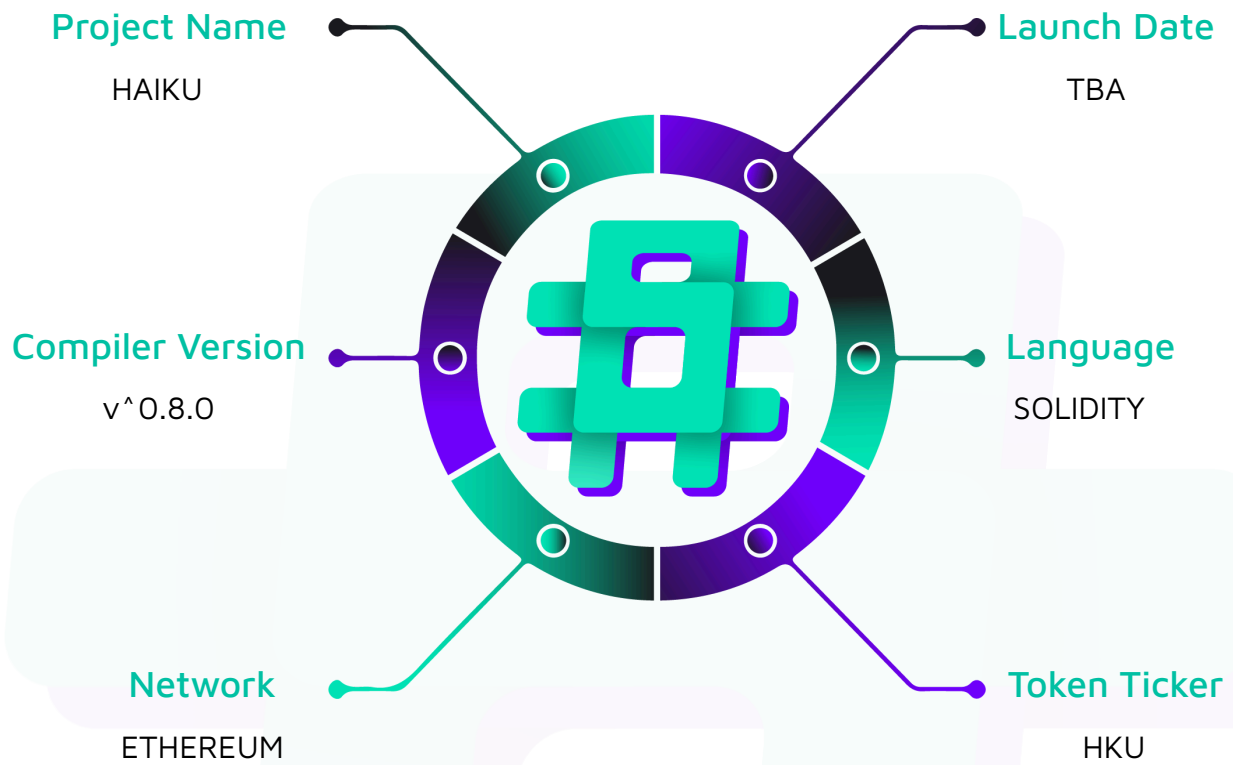
Project Type: DeFi

Compiler Version: ^0.8.0

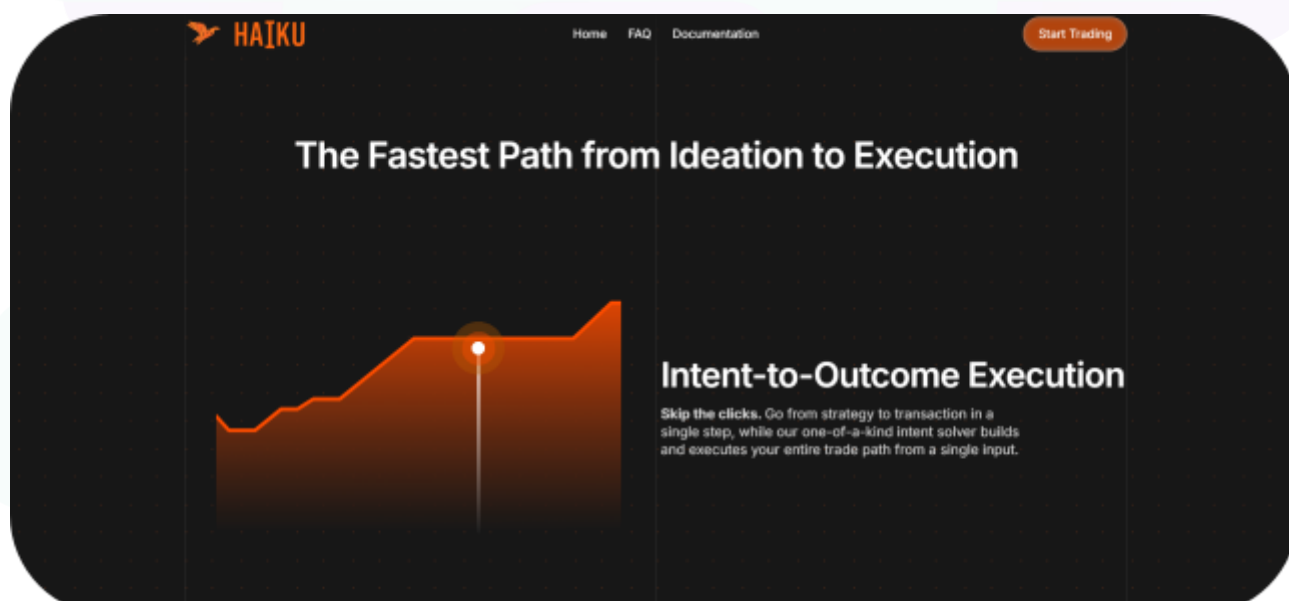
Website: <https://haiku.trade/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Haiku project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Haiku Smart Contracts
Platform	Ethereum / Solidity
Audit Date	September, 2025
Contract 1	AgentImplementation.sol
Contract 2	Router.sol
Contract 3	IAgent.sol
Contract 4	IRouter.sol
Contract 5	LayerZeroReceiver.sol
Contract 6	IPool.sol
Contract 7	OFTComposeMsgCodec.sol
Contract 8	DeployHyperEVM.s.sol
Contract 9	DeployLayerZeroReceiver.s.sol
Audited GitHub PR 1	https://github.com/Haiku-Trading/intent-swapper-contracts/pull/16
Audited GitHub PR 2	https://github.com/Haiku-Trading/intent-swapper-contracts/pull/17
Fix Review GitHub Commit Hash PR 1	30670d443bc718392eea16a7554082dfdb6ee3c0
Fix Review of GitHub Commit Hash PR 2	9de5fbb6065718809a8a11547611e99ea8f462b4

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

2 Medium severity vulnerabilities

3 Low severity vulnerabilities

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
LayerZeroReceiver.sol - Receives lzcompose() from OFTs and executes logics accordingly	Contract achieves this functionality.
Router.sol - Main entry point for all intent Execution	Contract achieves this functionality.
AgentImplementation.sol - Core logic for executing bundles of transactions	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Haiku project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Haiku project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] LayerZeroReceiver#constructor&setReserveGas - Unbounded ReserveGas Can Lead to Denial of Service

Description

The `constructor` and `setReserveGas()` functions allow the owner to set the `reserveGas` value without any upper bounds validation.

Impact

If an excessively high value is set for `reserveGas`, the check `gasleft() < reserveGas` within the `try/catch` block of the execution function could consistently fail, even for valid transactions with ample gas. This would prevent the `this.execute` call from ever succeeding, leading to a permanent denial-of-service for the protocol's core functionality.

Recommendation

Implement a sensible constant for `MAX_RESERVE_GAS` and validate the input against this cap in both the `constructor` and the `setReserveGas` function. This prevents the owner from accidentally or maliciously setting a value that could halt the protocol.

```
// Suggested fix

uint256 constant MAX_RESERVE_GAS = 100_000;

constructor(address _endpoint, address _router, address _owner, uint256 _reserveGas) {

    require(_reserveGas <= MAX_RESERVE_GAS, "Reserve gas too high");

    // ...

    reserveGas = _reserveGas;

    // ...
}
```

```
}  
  
function setReserveGas(uint256 amount) external onlyOwner {  
    require(amount <= MAX_RESERVE_GAS, "Reserve gas too high");  
  
    reserveGas = amount;  
  
    emit ReserveGasUpdated(amount);  
}
```

Status

Resolved

[M-02] LayerZeroReceiver#lzcompose - Gas Calculation Flaw May Cause Legitimate Transactions to Revert

Description

The contract attempts to forward gas to the `this.execute` call by pre-calculating `availableGas - reserveGas`. This approach can be brittle and may cause execution to fail even when sufficient gas is available.

Impact

Legitimate transactions that would otherwise have enough gas to succeed may fail because the gas forwarded to the `this.execute` call is prematurely reduced. The subsequent `catch` block logic may not accurately differentiate these failures from genuine out-of-gas errors, leading to unexpected reverts and a poor user experience.

Recommendation

A more robust pattern is to forward all `availableGas` to the sub-call and check the remaining gas *after* it returns or fails. This ensures the sub-call is given the maximum possible gas to complete its execution.

```
// Suggested Logic

uint256 gasBefore = gasleft();

try this.execute{gas: gasBefore}(token, amount, agentData, msg.value, receiver) {

    emit HaikuExecutionSuccessful(_guid);

} catch (bytes memory err) {

    uint256 gasAfter = gasleft();

    // Check if the remaining gas is below the reserve threshold

    if (err.length == 0 && gasAfter < reserveGas) {

        revert InsufficientGas(_guid);

    }

    // ... other error handling
```

```
}
```

Status

Resolved



Low

[L-01] LayerZeroReceiver&Router - Lack of Two-Step Ownership Transfer

Description

Both LayerZeroReceiver and Router contracts inherit from Openzeppelin's Ownable, which facilitates ownership transfer in a single transaction.

If the owner accidentally transfers ownership to an incorrect or non-existent address, all owner-privileged functions will be permanently locked. There is no recovery mechanism for such a mistake.

Recommendation

Replace Ownable with Openzeppelin's Ownable2Step. This implementation requires the proposed new owner to accept ownership in a second, separate transaction, effectively preventing accidental loss of ownership

Status

Resolved

[L-02] Router#addSigner - Missing Zero Address Validation for Signers

Description

Adding `address(0)` as a signer could potentially interfere with signature validation logic, as `'ecrecover'` returns `address(0)` on an invalid signature. This could allow improperly formed or forged signatures to be treated as valid

Recommendation

Adding a zero-address check at the beginning of the function.

Status

Resolved

[L-03] Router#rescue - Native Tokens are not Rescuable

Description

The rescue function is designed to recover accidentally sent ERC-20 tokens, but does not include functionality to rescue native tokens sent to the contract address.

Any native tokens accidentally sent to the Router contract will be permanently stuck, as there is no mechanism to withdraw them.

Recommendation

Implement a separate function, `rescueNative`, that allows the owner to withdraw a specified amount of the contract's native token balance.

Status

Resolved

QA

[Q-01] Contracts - Use of Floating Pragma Version

Description

The contract uses a floating pragma `^0.8.0`, this can lead to the contract being compiled with different compiler versions, potentially introducing subtle bugs or inconsistencies if a new compiler version with breaking changes is released.

Recommendation

It is best practice to lock the pragma to a specific, tested compiler version. This ensures deterministic and reproducible builds.

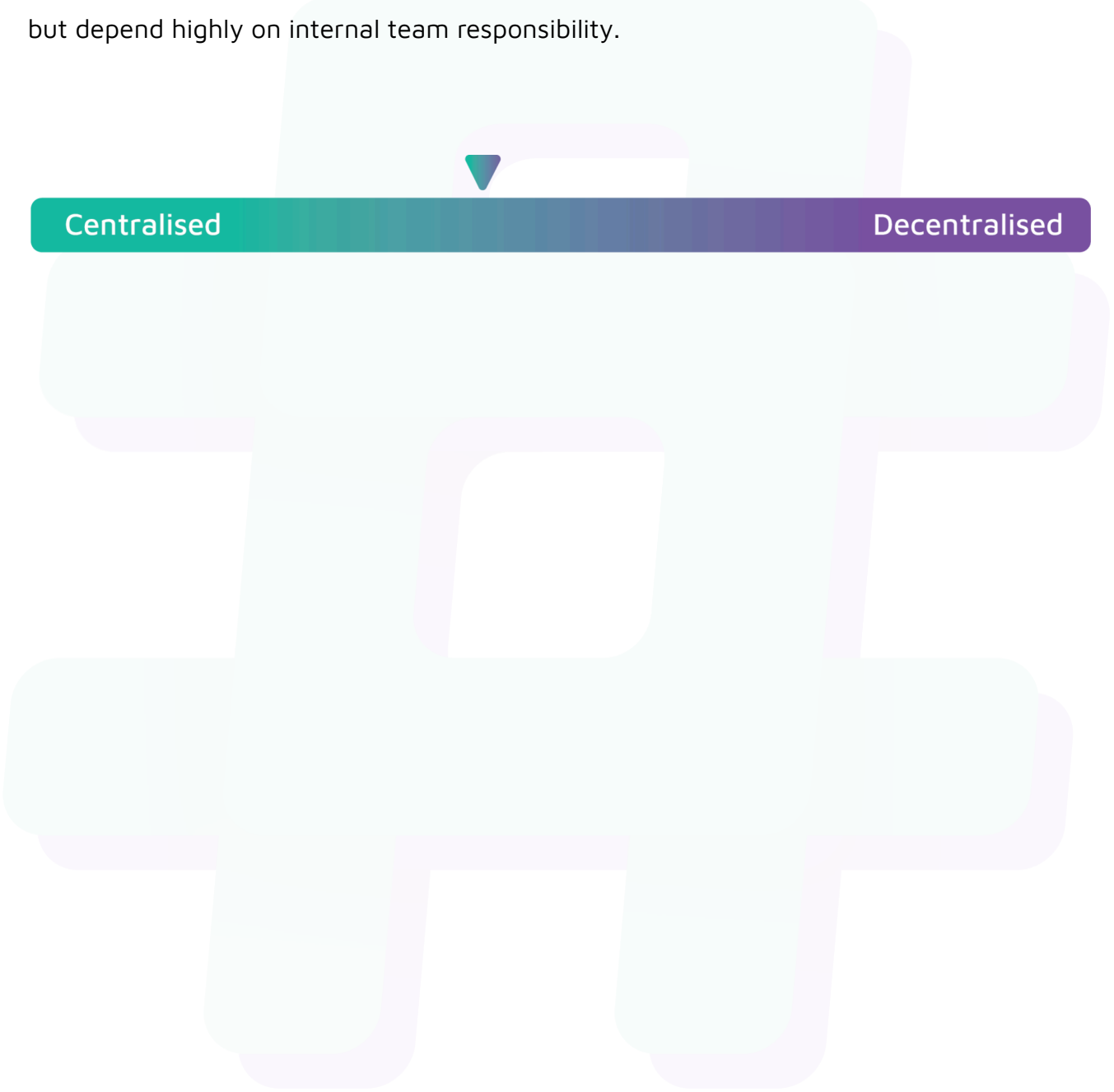
Status

Resolved

Centralisation

The Template project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Haiku project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.